WEEK 5 · Mon 19 Oct 2026

File Persistence, Validation, Sync/Async IO, File APIs and Assignment 1 Preparation

Complete the first realistic backend milestone by demonstrating DTO-to-domain transformation, file persistence, Java file APIs, synchronous versus asynchronous IO, validation, structured error handling and first tests.

Informatics Large Practical Project using Java & Spring Boot
Lecture deck — Main Lecture (core) + Additional Lecture (extended)
Colour key: blue = core/main · amber = extended/additional · slate = reading

MAIN LECTURE · CORE

Main Lecture

Course-structure focus (from §3, ILP Course Outline): File persistence, validation, sync/async IO, file APIs, test basics.

Purpose

Complete the first realistic backend milestone by demonstrating DTO-to-domain transformation, file persistence, Java file APIs, synchronous versus asynchronous IO, validation, structured error handling and first tests.

Topics

- DTO-to-domain transformation
- Jackson object mapping with `ObjectMapper`
- File-based JSON persistence
- Java `Path`
- Java `Paths`
- Java `File`
- Java `Files`
- Byte streams (`InputStream` / `OutputStream`) vs character streams (`Reader` / `Writer`)
- `BufferedReader` / `BufferedWriter` and the decorator (wrapper) pattern; charsets (`UTF_8`)
- `try-with-resources` and `AutoCloseable`
- Synchronous IO (blocking)
- Asynchronous IO: `Future`, `CompletableFuture`, `supplyAsync`, `thenApply`/`thenAccept`, `exceptionally`
- Thread pools / `Executor` and handling checked exceptions in async lambdas

Topics (cont.)

- Repository interfaces
- File repository implementation
- Configurable file paths
- Validation with Jakarta Bean Validation
- `@Valid`
- `@NotNull`
- `@NotBlank`
- `@Email`
- `@Min`
- `@PositiveOrZero`
- Global error handling
- Basic unit testing
- JUnit 5
- AssertJ

Topics (cont.)

- First service tests
- First mapper tests

Path represents a file-system path in the modern Java API.

```java
Path path = Path.of("/app/data/orders.json");
```

Files provides static methods for modern file operations.

```java
Files.createDirectories(path.getParent());

if (!Files.exists(path)) {
    Files.writeString(path, "[]");
}
```

File is the older file API and still appears in libraries.

```java
File file = path.toFile();

objectMapper.writeValue(file, orders);
```

Streams come in two families: byte streams and character streams, often wrapped in a Buffered….

Java I/O has two families.

Byte streams (`InputStream` / `OutputStream`) move raw bytes — images, zips, any binary.
Character streams (`Reader` / `Writer`) move *text* and apply a **charset** (always pass `UTF_8`).

A `Buffered…` class **decorates** another stream to batch disk access (and adds `readLine()` / `newLine()`);
an `InputStreamReader` / `OutputStreamWriter` bridges bytes ↔ characters.

Streams come in two families: byte streams and character streams, often wrapped in a Buffered…. (cont.)

JAVA

```java
// character I/O with buffering (text)
try (BufferedReader r = Files.newBufferedReader(path, StandardCharsets.UTF_8)) {
    String line;
    while ((line = r.readLine()) != null) {
        process(line);
    }
}
try (BufferedWriter w = Files.newBufferedWriter(path, StandardCharsets.UTF_8)) {
    w.write(json);
    w.newLine();
}

// byte I/O with buffering (binary)
try (var in = new BufferedInputStream(new FileInputStream(file))) {
    in.transferTo(out);
}
```

Streams come in two families: byte streams and character streams, often wrapped in a Buffered…. (cont.)

`try-with-resources` closes every stream automatically (they implement `AutoCloseable`).
For files, the modern `java.nio.file` API (`Files.readString` / `writeString` / `newBufferedReader`) is the high-level shortcut and should be preferred.

Synchronous IO blocks until the operation completes.

```java
String json = Files.readString(path);
Files.writeString(path, json);
```

Asynchronous IO allows work to continue while IO is pending.

```java
CompletableFuture<Void> future = CompletableFuture.runAsync(() -> {
    try {
        Files.writeString(path, json);
    } catch (IOException ex) {
        throw new UncheckedIOException(ex);
    }
});
```

Compose async results without blocking: supplyAsync, thenApply, thenAccept, exceptionally.

`CompletableFuture.supplyAsync(supplier)` runs the supplier on a worker thread and returns a handle **immediately**. Compose the result instead of waiting: `thenApply` transforms it, `thenAccept` consumes it, `thenCompose` chains another async call, `exceptionally` recovers from failure. Only `join()` / `get()` **block** the caller — keep those off hot threads.

JAVA

```
CompletableFuture<Path> f = CompletableFuture.supplyAsync(() -> exportAll(path));

f.thenApply(Path::toAbsolutePath)
 .thenAccept(p -> log.info("exported to {}", p))
 .exceptionally(ex -> { log.error("export failed", ex); return null; });

// run several exports concurrently, then wait once
CompletableFuture.allOf(f1, f2, f3).join();   // join() blocks — use sparingly
```

Async needs a thread pool and deliberate error handling.

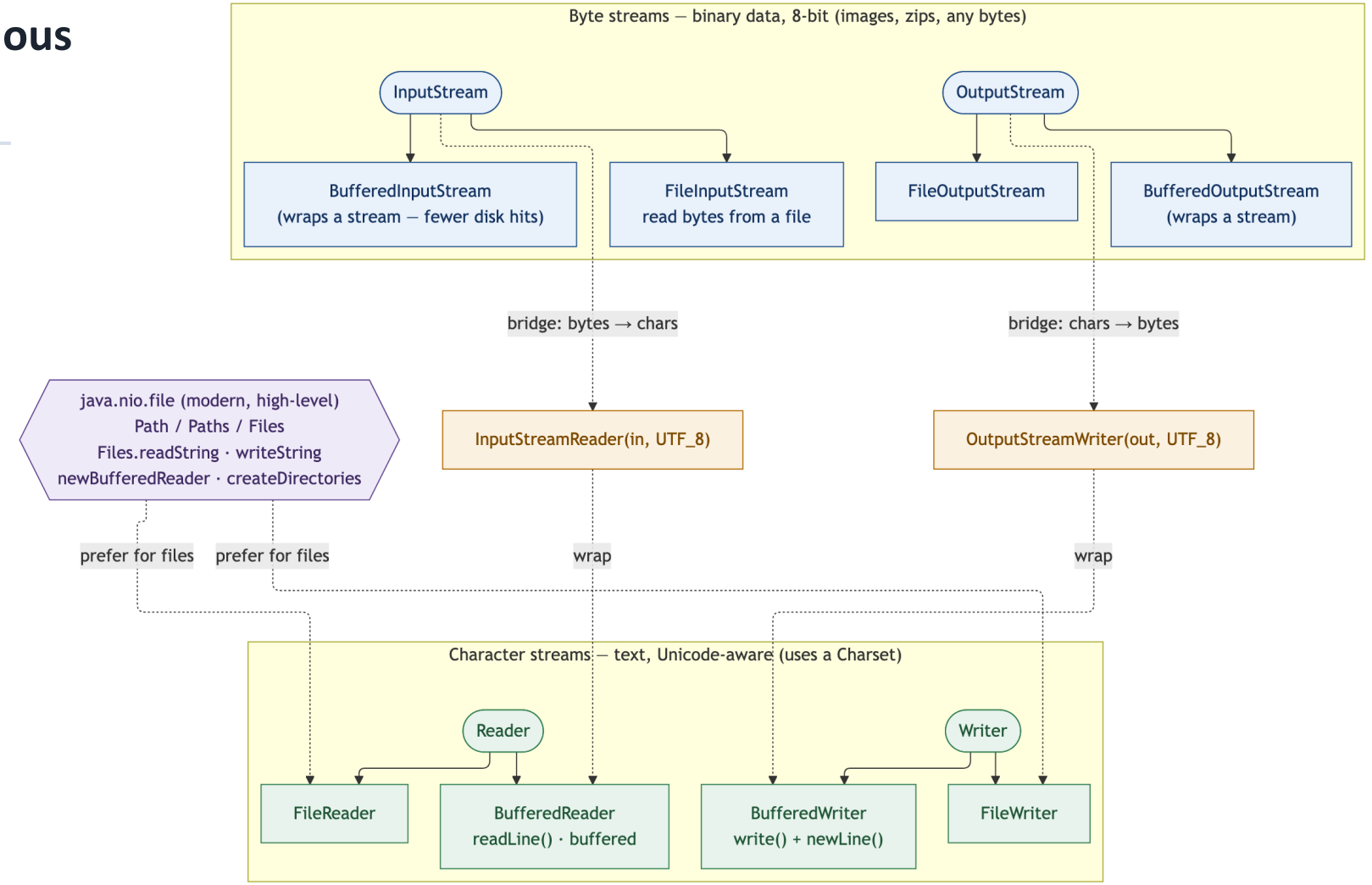
Work runs on the common `ForkJoinPool` by default, or on a custom `Executor` you pass as the second argument to `supplyAsync`. Because the functional types (`Supplier`, `Function`) do **not** declare checked exceptions, a checked `IOException` inside the lambda must be wrapped (e.g. in an `UncheckedIOException`) or handled there.

JAVA

```
ExecutorService io = Executors.newFixedThreadPool(4);
CompletableFuture<Path> f = CompletableFuture.supplyAsync(() -> {
    try {
        return writeAll(path);                // may throw IOException
    } catch (IOException ex) {
        throw new UncheckedIOException(ex);   // checked -> unchecked for the pipeline
    }
}, io);                                       // run on our executor, not the common pool
```

How file I/O and asynchronous work fit together

The I/O class map — two families, plus buffering and the `nio` shortcut.

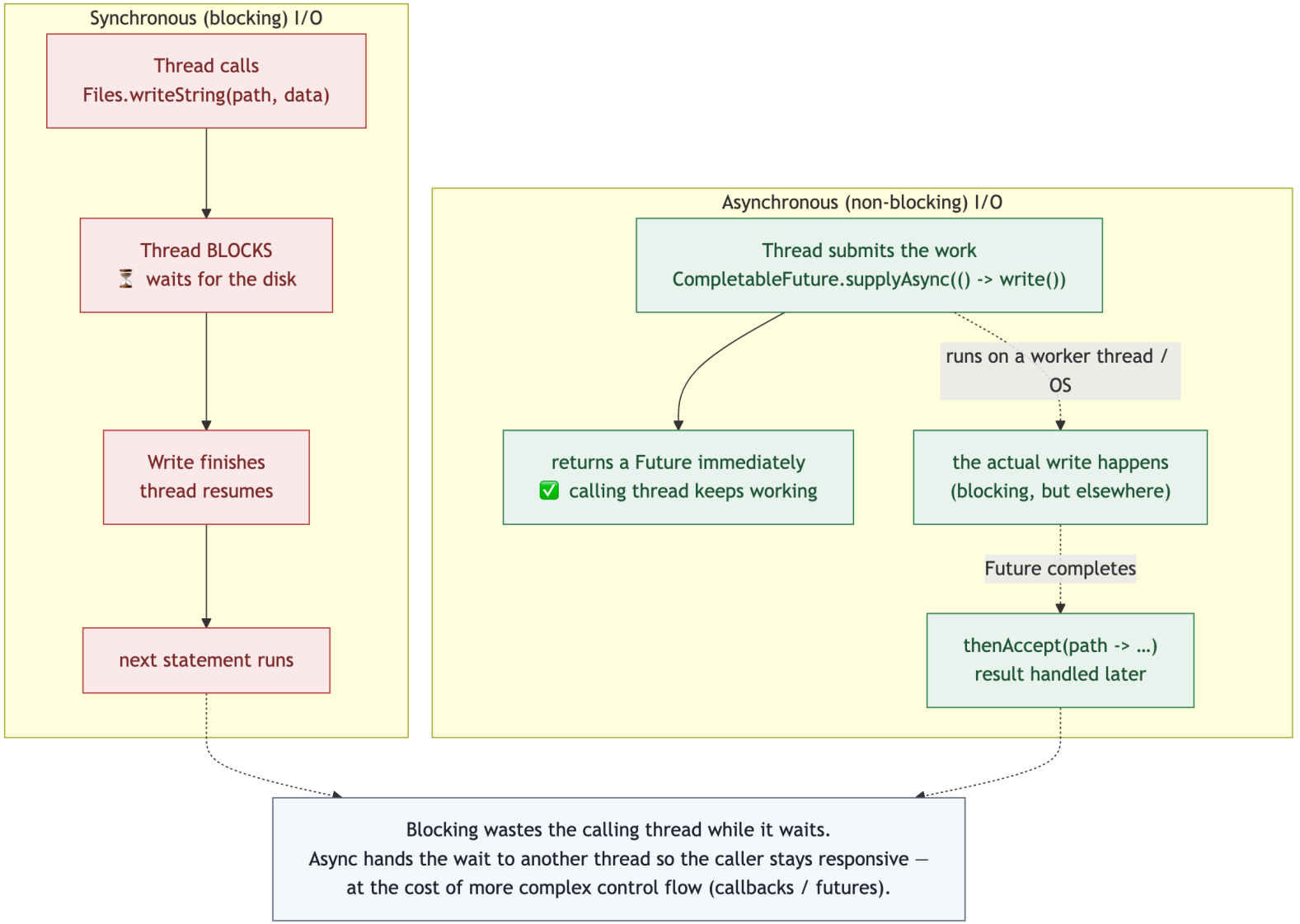


Java I/O classes: byte streams (InputStream/OutputStream, with FileInputStream/FileOutputStream and Buffered wrappers) versus character streams (Reader/Writer, with FileReader/FileWriter and BufferedReader/BufferedWriter); InputStreamReader and OutputStreamWriter bridge bytes and characters; and the modern java.nio.file Path/Files high-level API preferred for files.

How file I/O and asynchronous work fit together (cont.)

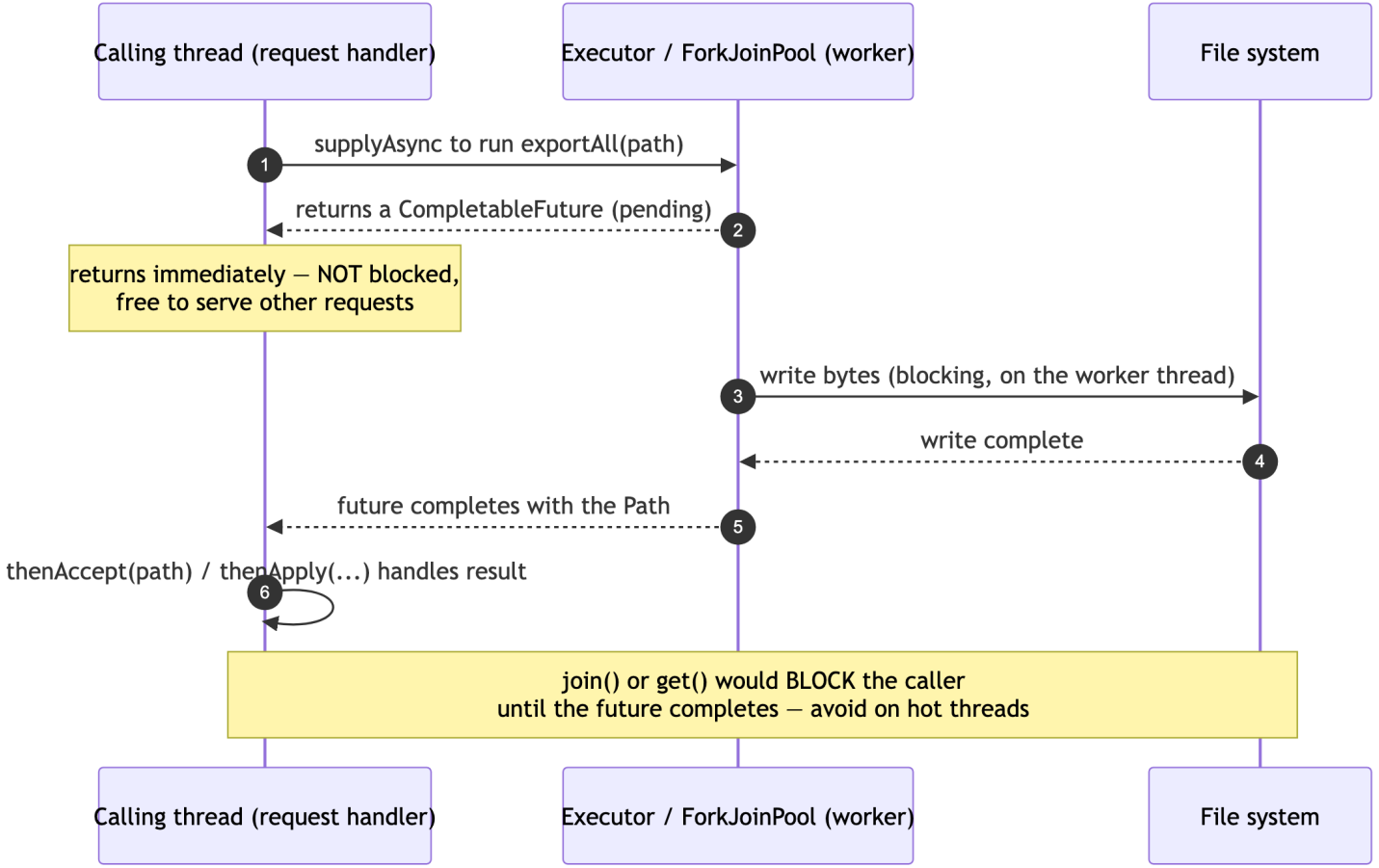
Synchronous vs asynchronous
— who waits, and where.

Synchronous blocking I/O, where the calling thread blocks waiting for the disk and then resumes, versus asynchronous non-blocking I/O, where the thread submits the work, receives a Future immediately and keeps working while the write runs on another thread, with the result handled later via a callback.



How file I/O and asynchronous work fit together (cont.)

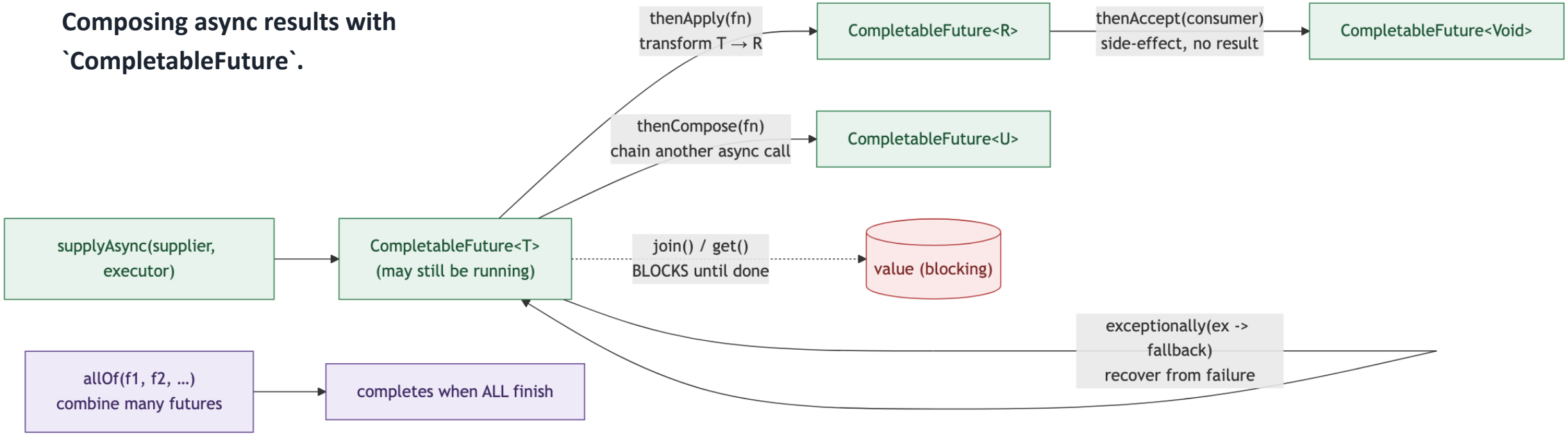
Async I/O over time — the thread hand-off.



Sequence of an asynchronous export: the calling thread calls supplyAsync and immediately receives a pending CompletableFuture; a worker thread performs the blocking write to the file system; when it completes, the future resolves with the Path and the caller handles it via thenAccept; calling join() or get() would block the caller until completion.

How file I/O and asynchronous work fit together (cont.)

Composing async results with `CompletableFuture`.



CompletableFuture composition: supplyAsync produces a CompletableFuture; thenApply transforms the value, thenAccept consumes it, thenCompose chains another asynchronous call and exceptionally recovers from failure; allOf combines many futures; join() or get() block until the value is ready.

Repository interfaces isolate persistence decisions. They are (by convention) used to access storage (persistance)

JAVA

```java
public interface OrderRepository {
    Order save(Order order);
    Optional<Order> findById(String id);
    List<Order> findAll();
}
```

ObjectMapper can persist lists of domain objects as JSON (helpful converting JSON to files and back).

```java
objectMapper.writeValue(path.toFile(), orders);

List<Order> loaded = objectMapper.readValue(
    path.toFile(),
    new TypeReference<List<Order>>() {}
);
```

Validation annotations express request rules (needed to validate requests and data).

```java
public record PartnerBuyerDto(
    @NotBlank String id,
    @NotBlank String fullName,
    @Email String email
) {}
```

ADDITIONAL LECTURE · EXTENDED

Additional Lecture

Course-structure focus (from §3, ILP Course Outline): Assignment 1 integration and submission deepening.

Purpose

Walk through Assignment 1 integration and submission expectations.

Topics

- Assignment 1 requirements
- Expected package structure
- Request/response flow
- File persistence troubleshooting
- Validation troubleshooting
- Logging and exception handling checklist
- Docker image export and load process
- README quality
- Tutor-style submission check

Loading, running and checking a container.

```bash
docker load -i campusshop-api-assignment1.tar
docker run -p 8080:8080 campusshop-api:assignment1
curl http://localhost:8080/actuator/health
```

Building a container image and saving it.

```bash
mvn clean package
docker build -t campusshop-api:assignment1 .
docker save campusshop-api:assignment1 -o campusshop-api-assignment1.tar
```

READING & FURTHER NOTES

Reading & Further Notes

Consolidate, then prepare for next week.

Homework for Next Lecture

What to do

Complete the week 5 lecture project in `student-app`:

- **File repository lookup:** `FileOrderRepository.findByCustomerId` returns only the stored orders of one customer.
- **File repository delete:** `FileOrderRepository.deleteById` removes the order, writes the file again and reports whether anything was removed.
- **Synchronous export:** `OrderExportService.exportAll` writes all orders to a file and creates the parent directory when needed.
- **Asynchronous export:** `OrderExportService.exportAllAsync` runs the same export on another thread with `CompletableFuture.supplyAsync` and returns at once.
- **Character I/O:** `TextFiles.readLines` reads a file line by line with a `BufferedReader` inside `try-with-resources`.

Run `mvn test` after each task until all tests are green.

Homework for Next Lecture (cont.)

Why it matters

These are the file techniques Coursework 1 relies on: a repository that keeps every order in one JSON file, reads and writes through `Path` and `Files`, and resources that are always closed. The two export methods show the difference between blocking the calling thread and handing the wait to another thread, which matters whenever a request has to do slow I/O.

Reading for Next Week

Topics — *Recommended:* Mockito basics, MockMvc basics, `@WebMvcTest`, `@SpringBootTest`, Arrange/Act/Assert. *Optional:* test naming conventions, the test pyramid, Spring Boot test slices.

Priority	Reading	Scope
Recommended	Cloud Native Spring in Action (Vitale), UoE library	section 3.4 Testing a RESTful application with Spring
Recommended	Martin Fowler: Mocks Aren't Stubs	article
Recommended	Spring Boot Reference: Testing Spring Boot Applications	"Auto-configured Spring MVC Tests" and "Mocking and Spying Beans"
Recommended	JUnit 5.11 User Guide	sections 1 and 2 (overview, writing tests)
Optional	Learning Spring Boot 3.0 (Turnquist), UoE library	ch. 5 Testing with Spring Boot
Optional	Ham Vocke: The Practical Test Pyramid	article
Optional	The Cucumber Book (Wynne), UoE library	ch. 1 to 3 (why Cucumber, first taste, Gherkin basics); the book's code is Ruby, these chapters are about Gherkin